

08 · Content Stewardship & Data-Maintenance Runbook

The [Deploy Runbook \(02\)](#) tells you how to run the **servers**. This document tells you how to keep the **data** correct and current — reviewing AI extractions, updating commitments, fixing rotted source links, handling “that’s wrong” complaints, and the routine chores that keep the RAP Index trustworthy after handoff.

It is honest about one thing throughout: **which tasks a non-technical steward can do in the browser, and which still require a developer** (the repo, a terminal, and AWS access). Several important maintenance jobs are, today, developer-only — that itself is on the [roadmap](#).

Companion documents: [01 · Project Audit](#), [02 · Deploy Runbook](#), [05 · Design & User Journeys](#), [07 · Data Governance](#), [09 · Product Roadmap](#). Glossary in the [README](#).

0 · The mental model: two separate data layers

The platform has **two independent bodies of data**, and a steward maintains them very differently. Confusing them is the most common mistake.

	The seeded RAP Index	The RAP-extraction data
What it is	~115 hand-curated commitments from public disclosures	AI-read commitments from uploaded RAP PDFs
Where users see it	/commitments, /organizations, /my-commitments	/extract, /my-rap
Where it’s edited	in code (<code>fixtures.ts</code>), then re-seeded to the database	in the browser (the review queue)
Who can edit it	a developer (needs repo + terminal + AWS)	an Institute curator (just logs in)

Keep this split in mind: **editing the seeded Index is a developer task; reviewing extractions is a curator task.**

Part A · The extraction review queue (the curator’s core loop)

This is the one job a non-technical Institute curator does directly, in the browser, and it is the heart of data quality. Every uploaded RAP flows through the same lifecycle:

Pending → Extracting → Pending Review → Confirmed (or Rejected; Failed on an error).

What auto-publishes vs. what needs you. A clean extraction — no validation issues, no disagreement, no flagged fields — publishes automatically and never appears in your queue. Anything with a flag lands in the review list at **/extract**, grouped into *In Progress*, *Failed*, and *Awaiting review*.

Reviewing a flagged document (per document): 1. Read each field with its **verbatim quote and page** shown beside it; flagged fields are highlighted. 2. **Resolve the Business Number** — type the org’s BN, or mark it “self-asserted.” 3. **Check off each flagged field** once you’ve confirmed it against the source (or correct it). 4. **Save & publish** — the button stays disabled until *every* flagged field is checked off **and** the BN is resolved. On publish, the platform writes the canonical org, RAP document, commitments, and baselines, and records exactly which fields you verified. 5. Or **Reject** with a reason.

One crucial scope note, taken straight from the code: “*This is extraction QA, NOT Indigenomics truth-verification.*” You are confirming **that the AI read the document correctly** — not that the company’s claim is true or its RAP is any good. That distinction protects the Institute: the platform reports what a company *said*, faithfully, with its source.

Part B · Updating the seeded RAP Index (a developer task)

The ~115 Index commitments live **in code** (`src/lib/commitments/fixtures.ts`), not in a database you can edit through a screen. They reach production by running a **seed script** that writes them into the live table.

The gotcha every steward must know — quoted from the code itself:

Editing `fixtures.ts` changes nothing a user sees until someone re-seeds the production database. The current file even carries a live example: three source URLs were fixed in the code on 2026-08-10 but are **“NOT yet in production ... until the seed data is redeployed/reseeded.”**

Two consequences:

- **Updating a commitment or a source link is a two-step, developer-only job:** edit `fixtures.ts` (and the matching row in [DATA_VERIFICATION.md](#)), then run the seed against the production stage (`npm run seed:sst`). It needs the repo, `tsx`, `sst shell`, and AWS credentials — **a non-technical steward cannot do it unaided.**
 - **Re-seeding is an *upsert*, not a replace.** It overwrites rows by key but does **not delete** a row you removed from the code. Deleting a commitment for good means deleting it from the database directly — there is no per-commitment delete tool for the Index today (see Part I).
-

Part C · Source-link maintenance (manual — there is no auto-checker)

Every seeded commitment carries a **source link** to the public page it came from, and **links rot**. There is **no automated link-checker** in the platform — every check to date has been a person sweeping the list by hand.

The register. [DATA_VERIFICATION.md](#) §4 is the human-maintained master list of all commitments with their source URLs and last-checked status.

What a link check looks like (this exact process just ran on 2026-08-10 and caught three dead links): 1. Walk the master list; HTTP-check each `source.url`. 2. **Some corporate/government sites block automated checks** (bot/geo protection) — those must be opened **in a real browser**, not trusted to a script. On the last sweep, two of three fixes (Sun Life, Alberta Health Services)

were bot-/geo-protected and had to be confirmed by hand. 3. Fix any rot in **both** `fixtures.ts` and `DATA_VERIFICATION.md`. 4. **Re-seed** so the fix reaches production (Part B).

A quarterly link sweep is a sensible cadence for a public index that names real organizations.

Part D · Corrections and data quality

Start from the disclaimer. Index figures are stated, in three places, as *“illustrative snapshots ... verify against the source before treating any number as exact.”* A “your number is wrong” query is answered first by pointing at the cited source and this framing.

The integrity convention. By deliberate curation, no seeded status ever exceeds **“reported”** — nothing is marked “confirmed,” because confirmation is exactly the independent-attestation layer the platform adds and which scraped public data doesn’t have. Note this is a **curation convention**, not a software guard: nothing in code stops a future editor from typing “confirmed,” so stewards must uphold it by hand.

How a correction is actually made today: - *A seeded commitment is wrong:* edit `fixtures.ts` → re-seed (Part B). (Remember: deletions need a direct database delete.) - *An AI-extracted record is wrong:* a developer can cascade-delete an organization and all its extracted data with `scripts/delete-org.ts`, or re-upload a corrected document.

Right-of-reply is a gap you should know about. A *claimed* company (logged in, BN-verified) can record its own progress and opt into the Index via `/my-rap`. But the ~100 **scraped** organizations have **no login and no channel** to dispute or annotate their entry — a complaint from one of them is handled entirely off-platform by a steward editing the fixtures. For a public index naming real companies, consider a documented correction/right-of-reply process (it’s on the [roadmap](#)).

Part E · Organization identity — Business Number curation

Organization identity is anchored to the 9-digit **CRA Business Number (BN)**. A curated crosswalk (`org-bn-map.ts`) maps `org` → BN for **37 of the 103** seeded orgs; its header warns “[!] *VERIFY BEFORE THE PROD MIGRATION,*” and the remaining 63 are documented (in `bn-curation-worksheet.md`) as *deliberately* not curated, with the reason for each (banks, crown corporations, universities, etc.).

Curating a new org = look up its BN from the correct registry, add a row, and run the validator (`scripts/validate-org-bn-map.ts`), which checks the BN’s format, that the key matches a real seeded org, and (when the live registry is activated) that the name plausibly matches. This is a registry-literate, developer-run task — and the live registry cross-check is a **stub** until activated (see 09).

At runtime, a RAP extraction **cannot be published until its BN is resolved** — the review UI enforces this (Part A).

Part F · The weekly notifications digest

The platform can email the Institute a **weekly overdue-and-at-risk milestone digest** across the Index.

- **Automatic:** a Monday-morning cron runs the digest **on production only** (dev/ca stages deliberately don't send stray emails).
 - **Manual:** a “**Generate & send now**” button at /notifications sends on demand, any stage.
 - **The catch:** email only actually goes out if Amazon SES is verified and the sender/recipient are configured for that stage; otherwise the digest is still recorded in-app but shows “**skipped.**” And because that configuration is set at deploy time, a deploy that forgets it **silently reverts to “skipped” with no warning.** A steward should confirm each Monday that the digest shows *sent*, not *skipped* — and if skipped, use the button and flag the config to whoever deploys.
-

Part G · Before real launch — demo-data hygiene

The platform ships pre-loaded with demo logins for showcasing. **Before real users touch it:**

- Run scripts/purge-demo-logins.ts (it is **dry-run by default** — nothing deletes without **--apply**) to remove the demo @demo accounts while **keeping institute@demo** for staff.
- **Rotate the shared demo password** (demo-portal-2026) and give institute@demo a real, private one.
- **Replace the sample fixture documents** in scripts/fixtures/ (they contain real copyright-sensitive RAP content used for testing).

This is covered as a checklist in [02 · Deploy Runbook §5.3](#).

Part H · The steward's cadence (grounded checklist)

Task	Cadence	Who	How
Clear the extraction review queue (confirm/reject flagged docs)	as uploads arrive / weekly	Curator (browser)	/extract (Part A)
Check Failed extractions; retry or dismiss	weekly	Curator (browser)	/extract
Confirm the Monday digest sent (not “skipped”)	weekly	Curator (browser)	/notifications (Part F)
Source-link sweep of the Index	quarterly	Developer + manual	DATA_ - VERIFICATION.md → fix → re-seed (Part C)

Task	Cadence	Who	How
Corrections (wrong figure / not-our-commitment)	on complaint	Developer	edit fixtures + re-seed, or delete-org (Part D)
BN curation for a new org	on new org	Developer	<code>org-bn-map.ts</code> + validator (Part E)
AWS cost check (AI/extraction/email spend)	monthly	Whoever owns the account	AWS Budgets — <i>no in-app tool</i> (Part I)
Demo purge + password rotation	once, before launch	Developer	<code>purge-demo-logins.ts</code> (Part G)

Part I · Missing tooling (so expectations are honest)

Today, a steward should be aware these do **not** exist and are on the [roadmap](#):

1. **No automated link-checker** — link rot is found only when someone looks (Part C).
2. **Re-seeding is developer-only and upsert-only** — a non-technical owner can't update the Index unaided, and deletions don't propagate by re-seeding (Part B).
3. **No right-of-reply for scraped organizations** — only claimed companies have a voice (Part D).
4. **Status integrity is a convention, not a code guard** (Part D).
5. **The live business-registry check is a stub** — the automated wrong-entity guard can't run until activated (Part E).
6. **The production digest email depends on SES config** and reverts to “skipped” silently if a deploy forgets it (Part F).
7. **No in-app cost monitoring** for the AI/extraction/email spend a steward now owns — set an AWS Budget alarm (see [01 · Project Audit](#)).

*This runbook reflects the repository as handed off; the mechanics it describes live in `src/lib/commitments/` (seed data), `src/app/extract/` and `src/lib/rap/` (the review workflow), `scripts/` (seed, purge, validate, delete), and `DATA_VERIFICATION.md` (the source register). The guiding principle: the platform's trustworthiness is a ***maintained*** property, not an automatic one — the review queue and the link register are the two habits that keep it honest.*